

Лабораторная работа №14

Мартынова Милана Александровна

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	8
5. Выводы	11
Список литературы	12

Список иллюстраций

1	Первый скрипт	9
2	Второй скрипт	10
3	Третий скрипт	10

Список таблиц

1. Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2. Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
3. Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

3. Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: — оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; — С-оболочка (или csh) — надстройка на оболочкой Борна, использующая С-подобный синтаксис команд с возможностью сохранения истории выполнения команд; — оболочка Корна (или ksh) — напоминает оболочку С, но операторы управления программой совместимы с операторами оболочки Борна; — BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек С и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна.

4. Выполнение лабораторной работы

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой ($> /dev/tty\#$, где $\#$ — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов. (рис. 1)


```

mamartynova@mamartynova:~$ cat l.sh
#!/bin/bash

SEMAPHORE_FILE="/tmp/semaphore.lock"
WAIT_TIME=10      # t1
USE_TIME=3         # t2

if [ $# -ge 1 ]; then
    WAIT_TIME=$1
fi
if [ $# -ge 2 ]; then
    USE_TIME=$2
fi

echo "Процесс $$ запущен. Ожидание ресурса максимум $WAIT_TIME сек."

for ((i=1; i<=WAIT_TIME; i++)); do
    if [ ! -f "$SEMAPHORE_FILE" ]; then
        # Захват ресурса
        echo "$$" > "$SEMAPHORE_FILE"
        echo "Процесс $$ захватил ресурс на $USE_TIME секунд."
        sleep $USE_TIME
        rm -f "$SEMAPHORE_FILE"
        echo "Процесс $$ освободил ресурс."
        exit 0
    else
        echo "Процесс $$: ресурс занят процессом $(cat $SEMAPHORE_FILE), ждём...
($i сек)"
        sleep 1
    fi
done

echo "Процесс $$: не дождался ресурса за $WAIT_TIME секунд."
exit 1

```

Рис. 1: Первый скрипт

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`. (рис. 2)

```

exit 1
mamartynova@mamartynova:~$ cat g.sh
#!/bin/bash

if [ $# -ne 1 ]; then
    echo "Использование: $0 <команда>"
    exit 1
fi

CMD=$1
MAN_DIR="/usr/share/man/man1"
MAN_FILE="$MAN_DIR/${CMD}.1.gz"

if [ -f "$MAN_FILE" ]; then
    echo "=== Справка для команды $CMD ==="
    zless "$MAN_FILE"
else
    echo "Справка для команды $CMD не найдена в $MAN_DIR"
    exit 1
fi

```

Рис. 2: Второй скрипт

- Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что \$RANDOM выдаёт псевдослучайные числа в диапазоне от 0 до 32767. (рис. 3)

```

mamartynova@mamartynova:~$ cat bt.sh
#!/bin/bash

LENGTH=20 # длина последовательности

for ((i=0; i<LENGTH; i++)); do
    # $RANDOM от 0 до 32767 → делим на 26 → остаток 0..25
    R=$(( RANDOM % 26 ))
    # 65 = ASCII 'A'
    CHAR=$(printf "\\$(printf '%03o' $((65 + R)))")
    echo -n "$CHAR"
done
echo # перевод строки

```

Рис. 3: Третий скрипт

5. Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

Список литературы